Software Engineering Department
Braude College of Engineering

SmartLeafDetection-Hybrid-Deep-Model-for-Tomato-Leaf-Disease-Detection

Final Project – Phase B (61998)

CODE: 26-1-D-6

By:
Ariel Laniado | Ariel.Laniado@e.braude.ac.il
Noy Ben Ezra | Noy.Ben.Ezra@e.braude.ac.il

Advisor:
Dr. Dan Lemberg | lembergdan@braude.ac.il
Mrs. Elena Kramer | elenak@braude.ac.il

GitHub Link: Smart Leaf Detection Hybrid model - GitHub

Contents

Abstract

Early detection of tomato leaf diseases is essential for maintaining crop health and yield, yet manual field inspection is time-consuming and impractical for large-scale monitoring. This project presents SmartLeafDetection, a UAV-based system for automated analysis of tomato fields using aerial video data. The implemented approach processes UAV videos by detecting tomato plants and individual leaves using a two-stage object detection strategy, tracking detected leaves across frames, and classifying leaf diseases using a convolutional neural network. To improve robustness under open-field conditions, classification results are aggregated temporally before inferring plant-level disease status. The system produces GPS-linked outputs that associate detected diseases with spatial plant locations and stores historical analysis results for later review. The implemented framework demonstrates a scalable and practical foundation for precision agriculture based on UAV-assisted computer vision.

1 Introduction

In today's world, characterized by continuous population growth and increasing pressure on global food systems, agriculture plays a critical role in ensuring food security and economic stability. As the demand for agricultural products continues to rise, the need for efficient, reliable, and sustainable crop management practices has become more significant. Within this context, tomato is among the most widely cultivated crops worldwide and holds substantial nutritional and economic importance.

Despite its importance, tomato cultivation is highly vulnerable to a wide range of diseases caused by pathogens such as bacteria, fungi, and viruses. These diseases can lead to substantial yield reduction, deterioration in fruit quality, and increased production costs. Foliar diseases may spread rapidly among neighboring plants, making early symptom detection and timely intervention critical for minimizing economic losses and maintaining stable crop productivity.

Traditionally, disease detection in agricultural fields has relied on manual scouting, where trained personnel visually inspect plants for discoloration, lesions, or signs of leaf degradation. While this approach can be effective on a small scale, it is labor intensive, time-consuming, and heavily dependent on the inspector's expertise. Moreover, manual field scouting limits the ability to provide consistent and continuous coverage across large agricultural areas, reducing its effectiveness for long-term and large-scale monitoring.

In recent years, advances in machine learning and computer vision have enabled automated approaches for agricultural monitoring, allowing rapid analysis of field imagery and early identification of disease symptoms. In parallel, growing interest in unmanned aerial vehicles (UAVs) has encouraged the adoption of aerial imaging for large scale data acquisition, providing broad field coverage and an aerial perspective that can reveal foliar patterns not easily observed from the ground.

When combined with deep learning-based image analysis, UAV imagery provides a strong technological foundation for automated monitoring systems for tomato crops. However, aerial imaging also introduces practical challenges: individual leaves often occupy only a small portion of the frame and may be partially occluded by overlapping vegetation, motion blur, or background clutter. These conditions highlight the need for robust ob-

ject detection, tracking, and classification methods capable of handling real-world field environments.

In response to these constraints, this project implements an automated UAV-based framework for tomato crop monitoring, designed to analyze small and partially occluded leaf targets in aerial video footage. The system combines deep learning-based plant and leaf detection, object tracking, leaf-level disease classification, temporal aggregation, and GPS-linked reporting to produce actionable outputs for precision agriculture. The remainder of this work presents the supporting background and related studies, describes the implemented architecture and end-to-end processing pipeline, and concludes with the evaluation of the developed system under realistic field conditions.

2 Background and related work

2.1 Precision agriculture and computer vision for disease detection

Precision agriculture aims to enhance crop management through site-specific monitoring and targeted intervention, rather than uniform treatment across entire fields. Within this context, computer vision and deep learning have emerged as key enabling technologies for identifying visually observable disease symptoms at scale. Prior work such as Kamilaris and Prenafeta (2018) demonstrates that vision-based approaches can efficiently process large volumes of imagery under variable field conditions, including changes in illumination, background complexity, and plant motion. As a result, visual monitoring systems are widely recognized in the literature as a practical foundation for automated plant health assessment and data-driven decision support in precision agriculture.

2.2 PlantVillage Datasets and real-world generalization challenges

PlantVillage is a widely used public dataset for plant leaf disease research, providing labeled images of healthy and diseased leaves acquired under controlled conditions. In Too et al. (2019) and Brahimi et al. (2018), deep learning models trained on PlantVillage dataset achieved high classification accuracy for leaf-level disease identification, demonstrating the effectiveness of CNN-based approaches under controlled imaging conditions. Nevertheless, the literature identifies a domain generalization challenge: models trained on PlantVillage data often experience degraded performance when applied to real-world field imagery, which is characterized by complex backgrounds, illumination variability, occlusions and scale changes. This gap underscores the limitations of controlled datasets for direct deployment in operational agricultural environments.

2.3 Object detection in agricultural imagery

Object detection is a fundamental task in agricultural computer vision, enabling the localization of relevant plant structures, including individual leaves, within complex visual frames. Unlike image-level classification, object detection identifies both the presence and spatial location of multiple targets, making it particularly suitable for agricultural imagery where each plant comprises multiple leaves and both appear at different spatial scales within a single frame.

2.3.1 YOLO-Based object detection models

YOLO-based architectures perform object detection by directly predicting bounding box coordinates and associated class probabilities from the input image. These predictions provide spatial localization of detected objects without relying on an explicit region proposal stage. The resulting bounding boxes can subsequently serve as localized regions of interest for downstream analysis in multi-stage processing pipelines.

2.3.2 Comparative studies and scientific justification for YOLOv11

In Ramos and Sappa (2025), modern object detectors were benchmarked under consistent conditions to evaluate detection accuracy, inference latency and robustness across varying imaging conditions. These evaluations are particularly relevant for agricultural scenarios, where small targets and complex backgrounds require models that balance precision with computational efficiency.

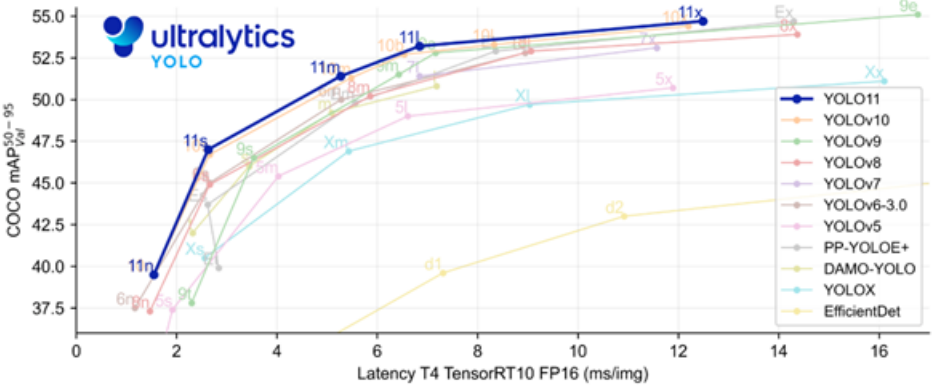

Figure 1: Comparison of object detectors' accuracy and latency

Based on these comparisons, YOLO-based detectors commonly show strong accuracy-latency trade-off, with newer versions generally improving upon earlier ones. As shown in Figure 1, YOLOv11 is positioned among the higher-accuracy models while maintaining low inference latency relative to other evaluated detectors, supporting its suitability for small object detection in agricultural imagery.

2.3.3 UAV field deployment challenges for YOLO-based detection

In UAV-based tomato field monitoring, object detection is challenging due to viewpoint variability, scale changes, background clutter, and high object density across frames. These difficulties are amplified in open-field conditions, where plants may appear small and partially occluded, making reliable localization difficult. In the broader literature, robust plant localization is commonly treated as a prerequisite for downstream leaf-level analysis, since it enables subsequent disease recognition to focus on relevant plant regions rather than the full scene. Lu et al. (2024) demonstrate that YOLO-based detectors can be effectively applied to UAV color imagery under real field conditions for robust plant localization, supporting the suitability of YOLO-based detection for field deployment scenarios.

2.4 Object tracking and identity in video analysis

In video-based agricultural analysis, independent frame-by-frame detection is insufficient for maintaining consistent object identities over time. Leaves and plants may undergo temporary occlusion, partial visibility or brief detection failures. Consequently, the literature emphasizes the integration of object tracking to associate detections across frames and preserve temporal continuity, which is essential for reliable video-based analysis.

2.4.1 SORT and DeepSORT

SORT is a widely used tracking framework that associates detections across frames using motion estimation and spatial overlap, offering an efficient baseline for multi-object tracking. However, its reliance on motion cues can lead to identity switches in the presence of occlusions. DeepSORT extends this approach by incorporating visual appearance features, improving identity preservation when objects are visually similar or temporarily obscured.

2.4.2 ByteTrack

ByteTrack further enhances tracking robustness by associating not only high-confidence detections but also lower-confidence bounding boxes. This strategy reduces track fragmentation in dense or cluttered scenes, where many objects are partially visible or weakly detected. The literature reports that this approach is particularly effective in environments such as agricultural imagery.

2.5 CNN-Based classification

CNNs are widely used for image-based classification tasks due to their ability to learn hierarchical visual representations from localized image regions. In detection-based pipelines, CNN classifiers are typically applied to regions of interest extracted from bounding boxes, enabling focused classification at the object level.

CNN	Accuracy	Model Size	Training Time
AlexNet	0.894	217 MB	1140.15 s
GoogLeNet	0.898	47.1 MB	332.228 s
VGG16	0.905	537.2 MB	1960.2 s
VGG19	0.903	558.4 MB	5411.31 s
ResNet-50	0.901	94.3 MB	2101.19 s
Inceptionv2	0.903	45.1 MB	2187.3 s
Inceptionv3	0.901	87.4 MB	6438.72 s
Inceptionv4	0.89	165 MB	5787.9 s
SENet	0.875	220.8 MB	1794.78 s

Table 1: CNN model accuracy and size comparison

2.5.1 ResNet50 as a Leaf disease classification

The ResNet family was introduced as a major advancement by enabling deeper networks through residual connections, which improve training stability and feature extraction. As shown in Table 1, ResNet50 achieves accuracy comparable to deeper CNN architectures while maintaining a substantially smaller model size. This balance between accuracy and

model complexity explains its frequent use of a baseline for leaf image classification, as also reflected in recent leaf disease studies that successfully employ ResNet-based classifiers Liang and Jiang (2023). Tomato specific extensions of ResNet50 have further been purposed in the literature to adapt this architecture to disease related visual patterns.

2.5.2 Transfer learning for tomato leaf diseases

In video-based analysis, visual appearance varies across frames due to motion, illumination changes and partial occlusions, increasing the difficulty of stable classification. Transfer learning using a pretrained CNN model enables robust feature extraction across such variability, making it well suited for tomato leaf disease classification in video processing scenarios.

In Saeed et al. (2023), transfer learning is applied to tomato leaf disease classification by fine-tuning pretrained CNN on tomato leaf dataset rather than training a network from scratch. The study reports improved performance when pretrained feature representations are adapted to disease related visual patterns such as lesions and texture changes.

2.6 Two-stage detection classification pipelines in the literature

In Lu et al. (2024) and Ramos and Sappa (2025), plant disease analysis is framed around a detection-classification paradigm in which spatial localization is treated as a necessary step before disease identification. These works address agricultural scenes that contain multiple plants and numerous leaves within a single image, making direct image-level classification impractical. Object detection is therefore used to isolate relevant plant structures, enabling subsequent disease-related analysis to be applied to localized regions rather than the full scene.

This separation between localization and classification is commonly adopted to manage scene complexity and reduce the influence of background content on disease recognition. By first constraining analysis to detected plant regions, classification methods can focus on visual cues that are more directly associated with disease symptoms.

2.7 Temporal aggregation in video-based analysis

In video-based plant analysis, the same leaf may receive different predictions across consecutive frames due to illumination changes, motion, partial occlusions and visual noise. Such frame-to-frame variability does not necessarily indicate changes in the underlying leaf condition but rather reflects transient fluctuations in visual appearance and detection quality.

To address this variability, video-based analysis commonly incorporates temporal decision aggregation, in which predictions are combined across a time window to derive a more stable entity-level outcome. By integrating probability estimates or classification decisions over multiple frames, temporal aggregation improves reliability and supports consistent disease inference, including scenarios involving multiple co-occurring diseases. This aggregation layer functions as a stabilizing stage that complements frame-level classification by mitigating short-term noise while preserving persistent disease-related evidence.

2.8 UAV-Based monitoring in precision agriculture

Integrating UAV platforms with deep learning models enables broad spatial coverage, high-resolution data collection and the extraction of insights that can be translated into targeted actions. Tsouros et al. (2019) describe a clear trend toward adopting this approach in precision agriculture, particularly for crop monitoring.

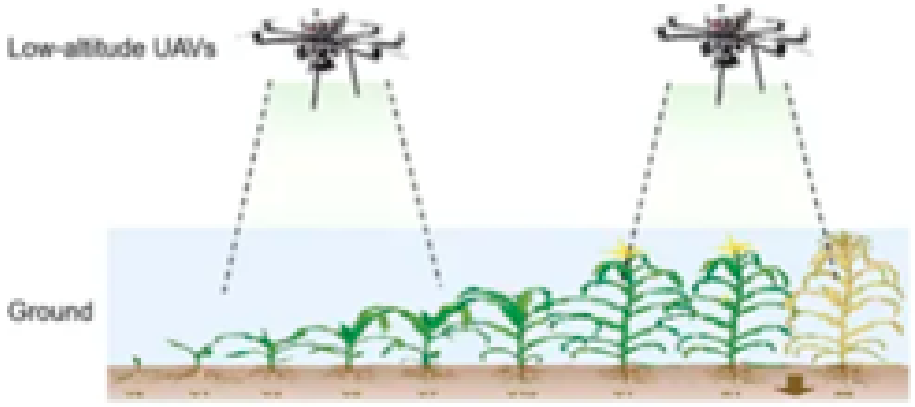

Figure 2: Illustrative of UAV-based crop monitoring

In precision agriculture, the highest value is achieved when the system produces operational outputs for decision support rather than detection alone. As emphasized by Tsouros et al. (2019), practical outputs include spatial analysis of affected areas. Accordingly, GPS-tagged acquisition plays an important role in translating diagnostic results into location-specific by indicating where intervention should be applied.

2.9 Summary of literature review

The reviewed literature indicates a consistent direction across key components of tomato leaf disease analysis. Controlled datasets such as PlantVillage enable string training for leaf classification, yet generalization to open-field conditions remain limited due to variability in illumination, occlusions and background complexity. YOLO-based object detectors are widely used as a foundation for isolating plants and leaves in multi-object agricultural scenes, with recent research support in the context of tomato leaf diseases. In video-based analysis, maintaining temporal consistency requires tracking and identity assignments to handle weak detections and partial occlusions. ResNet50 is repeatedly supported as a strong backbone for leaf disease classification, including tomato specific studies, while temporal aggregation across frames provides a stabilizing layer that improves decision reliability in sequential data. Finally, integrating UAV-based acquisition with geo-referenced localization supports the transition from research models toward practical precision agriculture decision support by linking diagnostic to spatially actionable treatment locations.

3 Project description

3.1 General Description

SmartLeafDetection is a UAV-based system developed for automated tomato leaf disease monitoring under real field conditions. The system analyses aerial drone footage to identify diseased tomato plants and support precision agriculture decision-making. In addition, the platform includes user management, historical flight storage and visualization of analysis results through a dedicated web interface.

3.1.1 System implementation and structure

The system comprises a web application, backend server, database layer, and an AI processing pipeline. The pipeline performs leaf detection with YOLOv11, stable leaf tracking with BoT-SORT and Global Motion Compensation, and tomato leaf disease classification with a dedicated ResNet50 classifier. Temporal aggregation stabilizes predictions during long UAV video analyses, and GPS/SRT telemetry links detections to field coordinates for hotspot mapping. Analysis results are stored in the database and presented in the client interface for visualization and historical flight management.

3.1.2 Target users

The system is designed for farmers, agronomists, and precision agriculture researchers who require efficient solutions for large-scale crop monitoring. It also supports UAV operators and agricultural organizations engaged in automated field analysis and early tomato disease detection.

3.2 System Architecture

The implemented system adopts a hybrid modular architecture that combines a client-side web application, a backend server, a database layer, and an AI processing module. This separation allows each part of the system to handle a specific responsibility, where the frontend manages user interaction, the backend coordinates requests and business logic, the database stores user data and analysis outputs, and the AI module performs the core video processing tasks. This architecture improves maintainability, scalability, and clarity of implementation for a system that integrates machine learning, data management, and web-based visualization. Within the AI component, the architecture follows a sequential processing pipeline. UAV video frames are first extracted and passed through leaf detection with YOLOv11, stable leaf tracking with BoT-SORT and Global Motion Compensation, disease classification with a dedicated ResNet50 classifier, temporal aggregation, and final leaf-level or plant-level decision making. GPS/SRT telemetry is then associated with the detected results to support field-level hotspot mapping and spatial analysis. This pipeline-oriented structure is appropriate because it preserves the logical order of the analysis steps and makes the system easier to test, extend, and debug. In addition, the use of a database and web interface enables persistent storage, historical flight review, and interactive visualization of disease detection results. The following figure presents the high-level architectural diagram of the implemented system and the relationships between its main components.

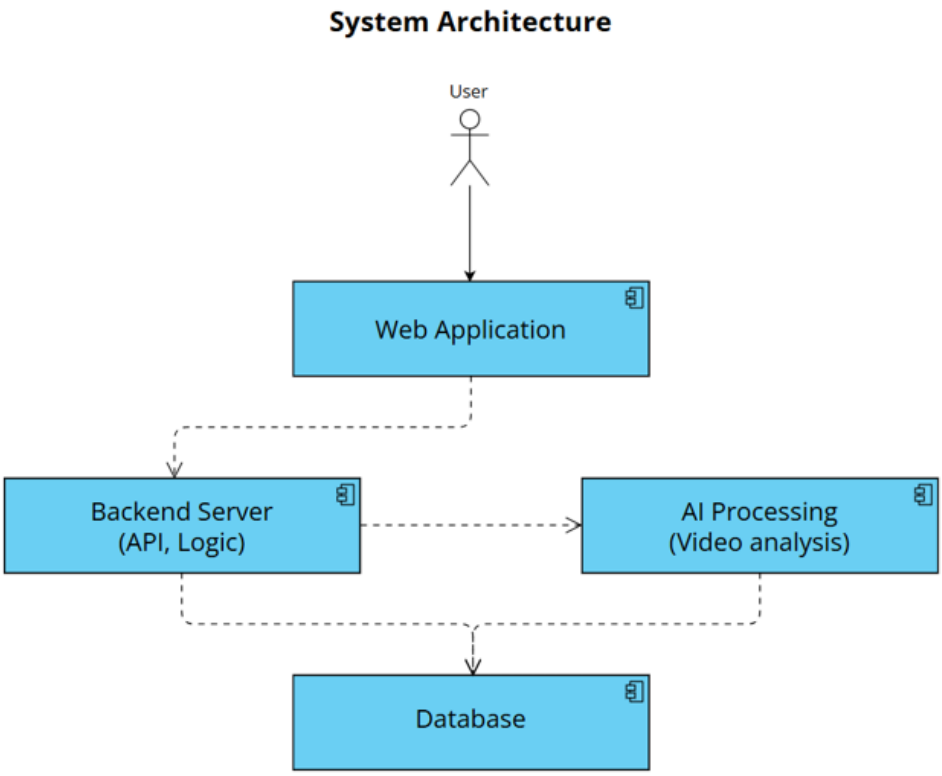

Figure 3: High Level System Architecture overview

3.2.1 Package diagram

The following figure presents the high-level package diagram of the implemented system and the relationships between its main components.

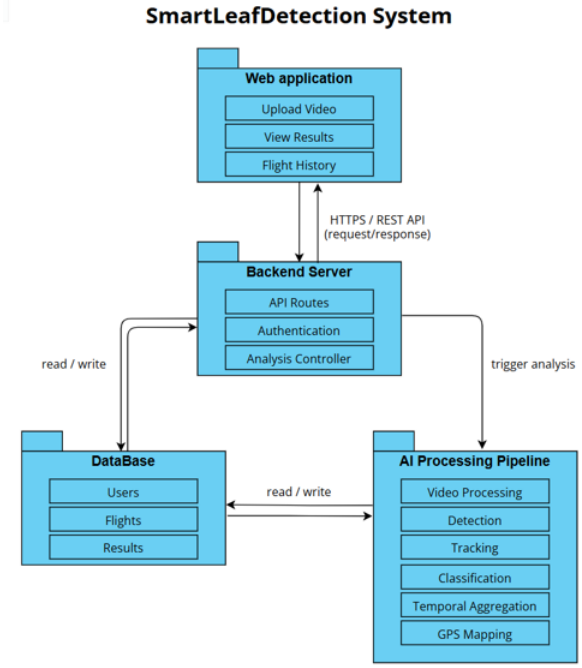

Figure 4: High Level Package Diagram

3.2.2 Activity diagram

The activity diagram presents the main execution flow of SmartLeafDetection, from UAV video upload to GPS-linked diagnosis storage and visualization. A more detailed internal activity diagram of the processing pipeline is provided in Appendix A.

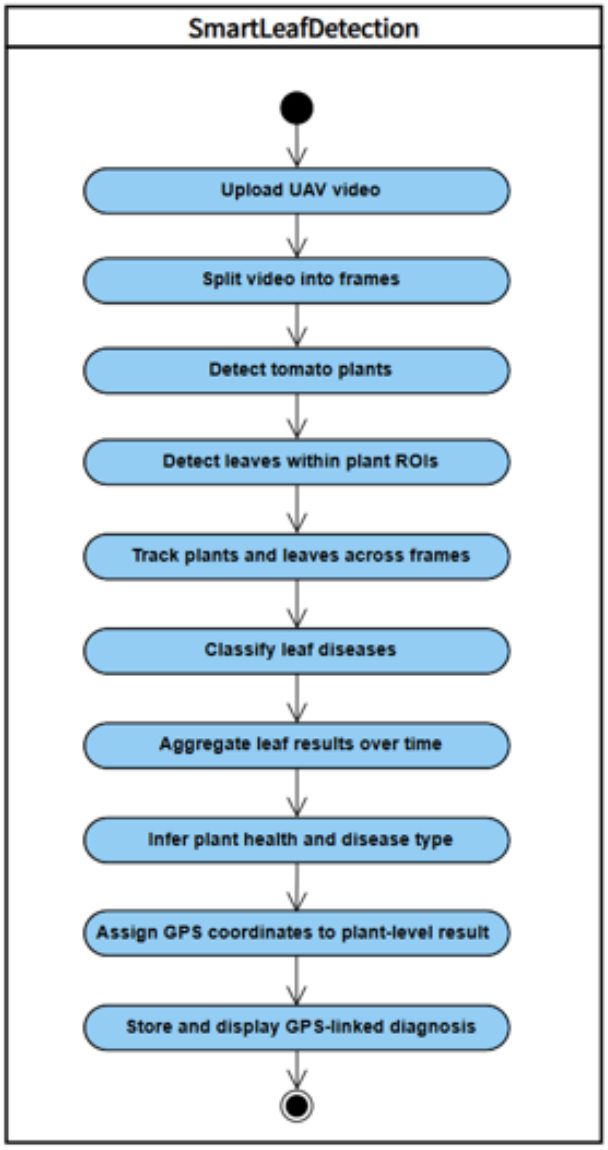

Figure 5: High Level Activity Diagram

3.2.3 Use-Case diagram

The use-case diagram presented in this section shows the primary interactions between the user and the system.

Figure 6: High Level Use-Case Diagram

3.3 Development process

3.3.1 Problem description

Detecting tomato leaf diseases under open-field UAV conditions presents several technical challenges. Leaves often appear as small objects within wide aerial frames, making reliable localization difficult. Disease symptoms may also be subtle and visually similar across different conditions, which complicates accurate leaf-level classification. In addition, UAV motion, lighting changes, shadows and partial occlusions can create inconsistent observations across frames, leading to unstable diagnostic decisions without tracking and temporal aggregation.

3.3.2 Dataset

The project used several datasets to support both classification and detection. The dedicated classifier was trained on a tomato-leaf disease classification dataset built from annotated leaf crops, while detection and tracking were supported by drone-frame and field-oriented datasets used under realistic agricultural conditions. The project also incorporates datasets such as Roboflow Tomato Diseases, CVAT Late Blight, PlantDoc, and Tomato-6K for detection and training experiments.

3.3.3 Database

The system uses a lightweight relational database implemented with SQLAlchemy and configured to use SQLite by default, with optional MySQL support. The database stores user credentials in the Users table and flight metadata in the Flights table, including video path, processing status, progress, frame counts, timestamps, and error information. Analysis outputs are stored in related tables: Frame Records for frame-level data, Plant Results for plant-level disease summaries, and Leaf Results for individual leaf detections

and crop references. This structure allows the system to manage users, track processing history, and retrieve analysis results for each uploaded flight.

3.3.4 Preprocessing

Video input is decomposed into sequential frames using OpenCV and processed frame-by-frame within the pipeline. The system supports configurable frame extraction modes, including every-N-frames sampling and target-FPS extraction, allowing adaptation to different video lengths and processing requirements. The pipeline also monitors processing speed against a 20 FPS target and issues a warning when performance falls below this threshold, supporting efficient and consistent frame handling throughout the workflow.

3.3.5 Development process stages

Leaf Detection using YOLOv11 The project uses YOLOv11 for leaf detection directly on UAV frames, rather than a two-stage plant detection followed by plant ROI cropping. In the implemented pipeline, the video frames are first processed to identify leaf regions, after which the detected leaves are tracked and passed to the disease classification stage. This leaf-centric strategy improves localization of small leaf objects, reduces background clutter, and supports more robust downstream analysis.

Tracking and identity assignment using ByteTrack Multi-object tracking is applied after leaf detection using BoT-SORT with Global Motion Compensation to associate leaf bounding boxes across consecutive frames. Each detected leaf is assigned a stable LeafID, and the tracking stage helps reduce identity switching, handle partial occlusions, and preserve entity continuity throughout the UAV video sequence.

ROI cropping and input normalization Leaf regions of interest are cropped from the detected leaf bounding boxes using a configurable padding factor, and the crop is clamped to the frame boundaries to avoid invalid regions. The cropped leaf ROIs are then resized to a fixed resolution of 224×224 pixels to provide a uniform input for the classifier. In addition, the images are converted from BGR to RGB, scaled to the range [0,1], and normalized using ImageNet mean and standard deviation values before classification. This preprocessing reduces background noise, standardizes the input representation, and lowers computational load, thereby enabling efficient processing of long video sequences.

Disease classification using ResNet50 A ResNet50-based convolutional neural network is applied using transfer learning to classify tomato leaf diseases at the ROI level. The classifier is trained on a dedicated leaf-disease dataset constructed from annotated leaf crops and evaluated using a validation split from the same processed dataset. The output of this stage is a probability vector over the canonical healthy and disease classes for each processed leaf ROI.

Temporal aggregation Classification outputs are accumulated for each tracked leaf across a predefined temporal window. For each LeafID, the system combines the per-frame observations using a confidence-weighted vote, and the label with the highest aggregated confidence is selected as the final leaf-level diagnosis. This stage reduces

frame-to-frame variability and improves the reliability of classification at the tracked-entity level.

Plant status inference Leaf-level results associated with the same PlantID are combined to infer the overall condition of the plant. If all tracked leaves are classified as healthy, the plant is labelled healthy, otherwise, it is labelled diseased, and the most relevant disease labels are reported based on their accumulated confidence. This stage produces a plant-level output record containing the plant identifier, final status, disease labels, leaf count, and diseased-leaf count.

GPS/SRT mapping and report generation Plant-level classification results are associated with approximate GPS coordinates derived from SRT telemetry files. The final exported report includes the plant identifier, disease labels, GPS coordinates, and evidence metrics to support field-level treatment and decision-making. When severity estimation is enabled, the report may also include a severity value for each record.

Web, backend, and database integration The backend, frontend, and database are integrated to support the full system workflow. FastAPI handles authentication, video upload, analysis initiation, and result retrieval, while SQLAlchemy stores users, flights, frame records, leaf results, and plant results. The frontend communicates with the backend through REST API requests and presents the processing workflow and analysis results to the user. This integration enables reliable processing, persistent storage, and traceability for each uploaded flight.

3.4 Challenges and solutions

The development of the UAV-based tomato leaf disease detection system involved several practical challenges caused by open-field conditions, video-based analysis, and software integration. The following section describes the main challenges encountered during implementation, including detection, tracking, classification, temporal consistency, and system-level integration, together with the solutions applied to address them.

3.4.1 Plant overlap and collision in dense field scenes

Neighboring plants can overlap in UAV frames, confusing associations. The system runs YOLOv11 leaf detection with BoT SORT to assign stable LeafIDs and reduce identity switching. Plant status is inferred by grouping tracked leaves by PlantID and aggregating leaf-level results.

3.4.2 Intermittent leaf visibility

Leaves may appear in some frames and become temporarily occluded or out of view in others, which can complicate tracking and temporal aggregation. The system addresses this by using tracking across frames and aggregating predictions over a temporal window instead of relying on a single frame.

3.4.3 Identity confusion due to leaf replacement over time

A leaf tracked in earlier frames may later be replaced by a visually similar leaf nearby, causing LeafID reuse. The system mitigates this using BoT SORT with Global Motion Compensation together with temporal aggregation, which reduces the impact of short-term identity errors on final plant level decisions.

3.4.4 Limited disease visibility due to leaf orientation

Disease symptoms may not always be visible due to leaf orientation, such as side-facing or folded leaves. The system reduces this limitation by aggregating classifications across multiple frames, increasing the chance of using clearer visual evidence when the leaf becomes more visible.

3.4.5 Motion blur and frame degradation

UAV motion, wind, or rapid camera movement may introduce motion blur and reduce frame quality, affecting detection and classification accuracy. The system mitigates this by processing video frame-by-frame and relying on temporal aggregation instead of isolated frame predictions.

3.4.6 Illumination variability in open-field conditions

Changing lighting conditions, shadows, and sunlight intensity create visual variability across frames, which may reduce detection and classification consistency. The system addresses this by using ROI-based classification and temporal aggregation to reduce the influence of unstable single-frame predictions.

3.4.7 Maintaining tracking consistency in long video sequences

Long video sequences can accumulate tracking errors, destabilizing PlantID and LeafID assignments. The system uses BoT SORT with Global Motion Compensation plus plant level temporal aggregation, so short tracking inconsistencies have limited impact on the final diagnosis.

3.4.8 Integration between AI pipeline, backend and database

Integrating the AI pipeline with the backend and database was challenging because video processing produces outputs at several levels, including frames, leaves, plants and flight records. The system addresses this by using FastAPI to manage upload and analysis requests, while SQLAlchemy stores users, flights and detection results in linked database tables for persistence and traceability.

3.5 Tools and technologies

3.5.1 Programming language and environment

The system is developed using Python, which provides extensive support for computer vision and deep learning workflows. Development and experimentation are conducted in a standard desktop environment using common Python tools and libraries.

3.5.2 Computer vision and video processing

OpenCV is used for video decoding, frame extraction and basic image processing operations. It enables sequential processing of UAV video data and supports efficient handling of long video sequences.

3.5.3 Deep learning frameworks

PyTorch is used as the primary learning framework for model implementation and inference. It supports integration of both detection and classification models within a unified processing pipeline.

3.5.4 Object detection and tracking

YOLOv11 is used for hierarchical detection of tomato plants in full UAV frames and tomato leaves within cropped plant regions. A multi-object approach is applied to maintain consistent LeafID and PlantID assignments across consecutive frames.

3.5.5 Dataset and model training resources

YOLOv11 detects tomato leaves directly in full UAV frames. Multi-object tracking uses BoT-SORT with Global Motion Compensation to maintain consistent LeafID and PlantID assignments across frames. A legacy plant detector exists, but the web application worker uses the leaf-centric pipeline.

3.5.6 Database management

A MySQL database is used to manage user accounts, flight history metadata and detection results. This supports structured storage of system data and retrieval of historical analysis records.

3.5.7 Visualization and output handling

Processed outputs, including detected disease labels and GPS-linked plant locations, are organized into structured results suitable for presentation and further analysis.

3.6 Engineering decisions and design considerations

Several engineering decisions were made to ensure reliable tomato leaf disease detection from UAV footage. The system uses a modular architecture that separates the frontend, backend, database, and AI pipeline, improving maintainability and allowing independent development and testing of each component.

A leaf-centric YOLO strategy was selected for detection. In production, leaves are detected directly in full UAV frames, which simplifies the runtime pipeline while maintaining effective localization of small targets in wide aerial scenes.

BoT-SORT with Global Motion Compensation is used to preserve object identity across frames. This supports stable LeafID and PlantID assignment, handles temporary occlusions, and links repeated observations of the same leaf over time.

For the classification stage, ResNet50 was selected because it provides a balance between classification performance and model complexity. Since the classifier receives cropped and normalized leaf ROIs, it focuses on disease-related features rather than irrelevant field background.

Temporal aggregation was added to improve prediction stability. Instead of relying on single-frame predictions, the system accumulates per-leaf classification probabilities over time before producing a final decision, reducing sensitivity to blur, lighting variation, and temporary misclassifications.

Finally, GPS/SRT mapping was integrated to make results operationally useful. By attaching approximate field coordinates to plant-level outputs, the system supports targeted treatment decisions and disease hotspot mapping.

3.7 Results and Conclusions

The developed system demonstrates that UAV-based tomato leaf disease analysis can be implemented as an end-to-end operational pipeline that combines detection, tracking, classification, temporal aggregation, and result visualization. In the implemented workflow, UAV video is processed frame-by-frame, tomato leaves are detected and tracked across time, and each tracked leaf is classified using the disease classification model. The results are then aggregated into stable leaf-level and plant-level outcomes that can be presented to the user through the system interface.

The system output shows that the proposed pipeline is capable of producing an organized analysis summary, including the distribution between healthy and diseased leaves, detected disease categories, confidence values, and representative cropped leaf images for inspected leaves. This makes the results understandable for the user and supports practical interpretation of the model predictions rather than presenting only raw detection outputs.

The results also emphasize the importance of tracking and temporal aggregation in video-based agricultural analysis. Since the same physical leaf may appear in several consecutive frames, the system does not treat each detection as a separate leaf. Instead, object tracking is used to maintain identity across frames, while temporal aggregation combines repeated observations before producing the final prediction. This approach reduces the effect of short-term errors caused by motion blur, partial occlusion, changes in leaf orientation, and illumination variability.

From an engineering perspective, the results confirm that the main system components were successfully integrated. The AI pipeline, backend services, database layer, and frontend interface work together to support video upload, processing, result storage, and visualization. This demonstrates that the project is not only a standalone model experiment, but a functional software system designed for agricultural monitoring workflows.

In conclusion, the project achieves its main objective of building a practical framework for tomato leaf disease analysis from UAV footage. The implemented system demonstrates the feasibility of combining YOLO-based detection, object tracking, ResNet50-based classification, and temporal aggregation for precision agriculture. Future improvements may include further training of the leaf detection model using additional annotated frames,

broader testing on additional field videos, quantitative evaluation against manually annotated ground truth, and optimization for faster large-scale deployment.

3.8 Project Reflection and Evaluation

This section reflects on the development process of the SmartLeafDetection system and evaluates the implemented pipeline in relation to the project objectives.

3.8.1 Lessons learned

The main lesson learned from the project is the importance of modular architecture. Separating the YOLOv11-based detection stage from the ResNet50-based classification stage made the system easier to develop, test, and integrate with the web application.

Another important lesson is that data preparation has a major effect on system behavior. Proper cropping, resizing, normalization, and validation of input images were essential for producing more stable model outputs.

The project also emphasized the value of temporal context in video-based analysis. Instead of relying on isolated frames, tracking and aggregation allow the system to combine repeated observations of the same leaf and produce more reliable final predictions.

3.8.2 Evaluation and Metrics

The evaluation focused on the disease classification component and its suitability for integration into the complete SmartLeafDetection pipeline. Since the classifier is applied to cropped leaf regions after YOLO-based detection, the selected model must provide a balance between classification accuracy, Macro-F1, inference time, number of parameters, and model size.

To evaluate the classification stage, three CNN architectures were compared under the same classification task: ResNet50, EfficientNetV2-S, and MobileNetV3-Large. All three models were trained and evaluated using the same dataset splits and evaluation process.

Model	Accuracy	Macro-F1	Macro Precision	Macro Recall	Params	Size	Inference
ResNet50 (production)	**94.2%**	**84.9%**	85.9%	89.6%	23.5 M	90.1 MB	14.0 ms
EfficientNetV2-S	95.3%	86.1%	89.0%	90.3%	20.2 M	77.9 MB	31.8 ms
MobileNetV3-Large	93.0%	82.6%	83.5%	87.9%	4.2 M	16.3 MB	13.3 ms

Table 2: Comparison of Disease Classification Architectures

The comparison shows that EfficientNetV2-S achieved the highest accuracy and Macro-F1, while MobileNetV3-Large achieved the smallest model size and fastest inference time. However, ResNet50 was selected as the production classifier because its accuracy and Macro-F1 were close to EfficientNetV2-S, while its inference time was significantly faster.

This makes ResNet50 a suitable choice for the implemented pipeline, where many cropped leaf images must be classified efficiently during video processing.

The evaluation also showed that ResNet50 performed strongly on the common high-support classes, including bacterial spot, early blight, late blight, healthy, and yellow leaf curl virus. The main limitation was the `target_spot` class, which had lower recall and was often confused with `septoria_leaf_spot` and `powdery_mildew`. Since this weakness appeared across the tested architectures, it suggests that the issue is mainly related to data limitations and visual similarity between disease symptoms, rather than a specific implementation problem.

References

1. Brahimi, M., Boukhalfa, K., & Moussaoui, A. (2017). Deep learning for tomato diseases: classification and symptoms visualization. *Applied Artificial Intelligence*, 31(4), 299-315.

2. Kamilaris, A., & Prenafeta-Boldú, F. X. (2018). Deep learning in agriculture: A survey. *Computers and Electronics in Agriculture*, 147, 70-90.

3. Liang, J., & Jiang, W. (2023). A ResNet50-DPA model for tomato leaf disease identification. *Frontiers in Plant Science*, 14, 1258658.

4. Lu, C., Nnadozie, E., Camenzind, M. P., Hu, Y., & Yu, K. (2024). Maize plant detection using UAV-based RGB imaging and YOLOv5. *Frontiers in Plant Science*, 14, 1274813.

5. Ramos, L. T., & Sappa, A. D. (2025). A comprehensive analysis of YOLO architectures for tomato leaf disease identification. *Scientific Reports*, 15(1), 26890.

6. Saeed, A., Abdel-Aziz, A. A., Mossad, A., Abdelhamid, M. A., Alkhaled, A. Y., & Mayhoub, M. (2023). Smart detection of tomato leaf diseases using transfer learning-based convolutional neural networks. *Agriculture*, 13(1), 139.

7. Too, E. C., Yujian, L., Njuki, S., & Yingchun, L. (2019). A comparative study of fine-tuning deep learning models for plant disease identification. *Computers and Electronics in Agriculture*, 161, 272-279.

8. Tsouros, D. C., Bibi, S., & Sarigiannidis, P. G. (2019). A review on UAV-based applications for precision agriculture. *Information*, 10(11), 349.

Appendices

Appendix A: Extended activity diagram

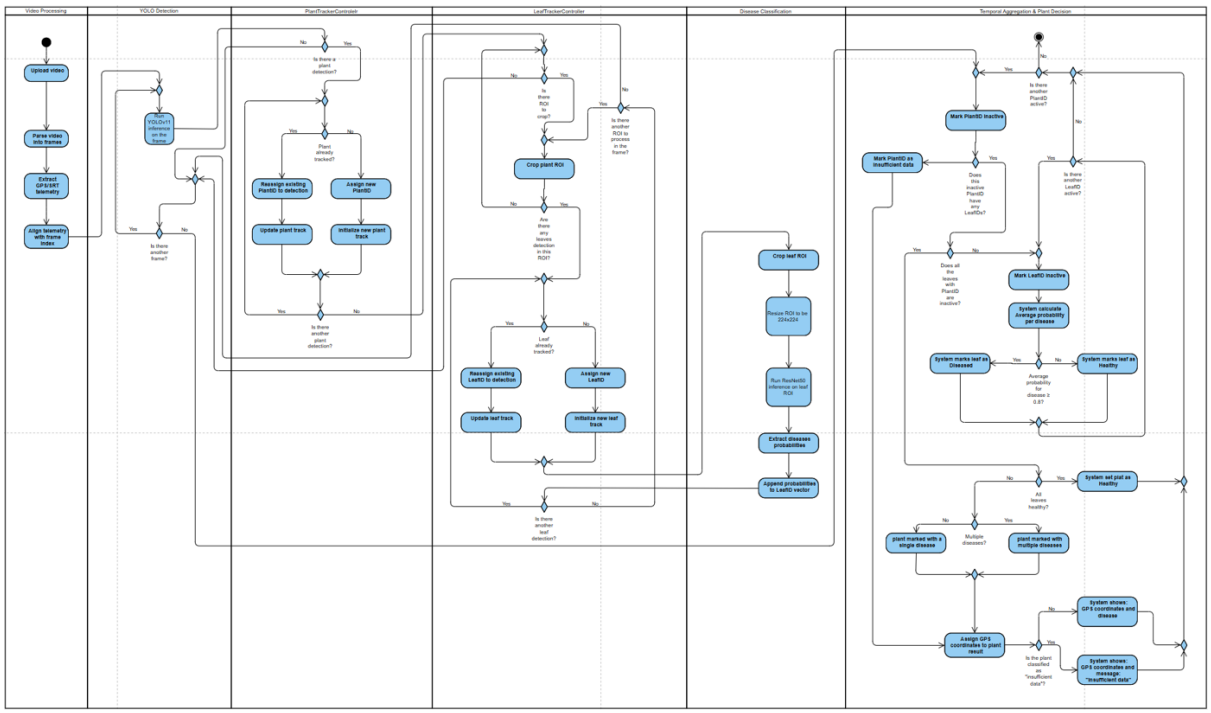

Figure 7: Activity Diagram

Appendix A presents the extended activity diagram of the proposed system, detailing the control flow across modules for plat/leaf detection, tracking, classification and temporal aggregation. This diagram complements the simplified activity diagram presented in 3.2.2.

Appendix B: User Guide

B.1 System overview

The SmartLeafDetection system is a web-based platform designed to detect and classify tomato leaf diseases from drone-captured video footage. The user uploads a video of tomato plants, and the system analyzes the footage to identify individual leaves, classify their condition, and present the results through the web application.

The system provides detection results, disease labels, confidence scores, and supporting visual evidence such as annotated frames and cropped leaf images. These outputs help the user review the health condition of the analyzed tomato plants and support early disease identification.

B.2 Accessing the system

Registration: When the user first accesses the system, the login page is displayed. If the user does not have an account, they can select the registration option and create a

new account by entering a username and password. After completing the registration process, the account becomes available for use.

Login: Existing users can log in by entering their username and password on the login page. After successful login, the user is directed to the main dashboard, where the main system actions are available.

Dashboard Overview: The dashboard is the main screen of the system. From this screen, the user can upload a new video, start a new analysis, view the status of existing analyses, and open previous analysis results. The dashboard allows the user to manage video analyses in one central location.

B.3 Main workflow

B.3.1 Uploading video To begin a new analysis, the user selects the upload option from the dashboard. The user then chooses a video file from their computer and uploads it to the system.

The uploaded video should clearly show tomato plants and leaves. Clear and stable footage improves the ability of the system to detect leaves and classify their condition correctly. After the upload is completed, the system creates a new analysis entry for the uploaded video.

B.3.2 Starting the analysis After the video has been uploaded successfully, the user starts the analysis by clicking the analysis button. The system then begins processing the video automatically.

During the analysis, the system extracts frames from the video, detects visible leaves, tracks leaves across frames, classifies their condition, and combines the observations into final results. Processing time may vary depending on the length of the video, video quality, and system performance.

B.3.3 Monitoring progress While the analysis is running, the system displays the current processing status and a progress indication. This allows the user to understand that the video is being analyzed and that the process is still active. The user can wait until the analysis is completed and then return to the results screen. When the analysis is finished, the status changes to complete, and the user can open the final results.

B.3.4 Reviewing results After the analysis is complete, the user can open the results page. This page presents the findings of the system in a visual and organized way. The results may include detected leaves, disease classifications, confidence scores, annotated frames, cropped leaf images, and summary information about the analyzed video. These results allow the user to review which leaves were identified, what condition was assigned to each leaf, and what visual evidence supports the classification.

B.4 Understanding the results

Disease Label: Each detected leaf is assigned a disease label by the system. The label represents the condition that the model identified for that leaf. For example, a leaf may

be classified as healthy or as showing signs of a specific tomato leaf disease. The disease label helps the user understand which diseases may be present in the analyzed crop. However, the system output should be treated as decision-support information and not as a replacement for professional agricultural judgment.

Confidence Score: Each disease label is shown with a confidence score. This score represents how certain the system was when assigning the classification. A higher confidence score means the system was more confident in its prediction. The confidence score should be used as supporting information when reviewing the results. It does not guarantee that the classification is correct in every case. For important decisions, the user should also review the supporting images and consider professional agronomic verification.

Detected Leaves and Bounding Boxes: The results screen may display bounding boxes around detected leaves in the video frames. A bounding box is a rectangular outline that marks where the system identified a leaf. These boxes help the user see which parts of the frame were analyzed. They also allow the user to check whether the system detected the correct leaf areas and whether any important leaves were missed.

Supporting Frames and Cropped Images: The system provides visual evidence for the analysis. This may include annotated frames from the original video and cropped images of individual leaves. The original frame shows the detected leaf in its surrounding plant context, while the cropped image focuses on the specific leaf region that was analyzed. These images allow the user to visually review the system's classification and compare it with the actual appearance of the leaf.

Summary Statistics: The results page may also include summary information about the analysis. This can include the total number of detected leaves, the number of leaves classified as healthy or diseased, and the distribution of disease labels. These statistics provide a general overview of the crop health condition in the analyzed video and help the user understand the overall result of the analysis.

B.5 Analysis history

The system keeps a history of uploaded videos and previous analyses. From the dashboard, the user can view past analyses and open their results again without uploading the same video. Each analysis entry may show information such as the video name, upload date, processing status, and access to the results page. This feature allows the user to review previous findings, compare analyses over time, and keep track of crop monitoring activity.

B.6 Tips for best results

To improve the quality of the analysis, the user should upload videos that clearly show tomato leaves. The following recommendations can help improve detection and classification performance.

Video Quality: The video should be clear, sharp, and recorded at a good resolution. Blurry or low-quality footage can make it difficult for the system to detect leaves and identify disease symptoms. The camera lens should be clean and focused before recording.

Lighting Conditions: The video should be recorded in good lighting conditions. Natural daylight is recommended, but very harsh shadows or very dim lighting should be avoided. Poor lighting may affect the appearance of leaf color and texture, which can reduce classification accuracy.

Camera Movement: The drone should move steadily and avoid sudden movements. Fast movement or shaking can create motion blur, making leaves harder to detect. A stable recording improves the quality of the analyzed frames.

Leaf Visibility: The tomato leaves should be clearly visible in the video. Footage recorded from too far away, or footage where leaves are hidden by shadows or overlapping plants, may reduce the quality of the analysis. The video should include enough visible leaf area for the system to process.

Recording Angle: The video should capture the plants from an angle that shows the leaves clearly. A combination of overhead and slightly angled views may provide better visual information than a video where leaves are barely visible.

B.7 Common user issues

Video Does Not Upload: If the video does not upload, the file format or file size may be the cause. The user should make sure that the selected file is a common video format and that the internet connection is stable. If the upload still fails, the user can try uploading a shorter or compressed version of the video.

Analysis Takes a Long Time: Processing time can vary depending on video length, video quality, and system performance. Longer videos usually require more processing time. If the analysis is still running, the user can wait and check the status again later.

Few or No Leaves Are Detected: If the system detects only a few leaves, the video may not show the plants clearly enough. This can happen when the footage is blurry, too far from the plants, too dark, or captured with fast camera movement. In this case, the user should record a clearer video with more visible leaves.

Results Look Inaccurate: If the results appear inaccurate or the confidence scores are low, the user should review the supporting frames and cropped leaf images. Inaccurate results may occur when the video quality is low, the leaves are partially hidden, lighting conditions are difficult, or the disease symptoms are unclear.

Appendix C: Maintainer Guide

C.1 Purpose of the appendix

This appendix provides the technical maintainer guide for the SmartLeafDetection system. Its purpose is to support the person responsible for running, configuring, maintaining, and extending the system after handover.

The appendix summarizes the system structure, main technologies, model files, setup process, execution instructions, configuration options, database handling, common maintenance tasks, and known system limitations.

C.2 What the system does

The system takes a drone video of a tomato field and produces a per-leaf disease report. It runs a four-stage pipeline:

1. **Frame extraction** – sample frames from the video and keep the sharpest, most leaf-dense segment.
2. **Leaf detection and tracking** – a YOLOv11 detector locates each leaf. A tracker, BoT-SORT, follows the same physical leaf across frames with a stable ID.
3. **Disease classification** – each leaf crop is classified by a dedicated ResNet50 image classifier.
4. **Temporal aggregation** – all observations of one tracked leaf are combined with a confidence-weighted vote into a single final diagnosis.

The result is delivered through a web application.

C.3 Technology stack

Layer	Technology
Leaf detection / tracking	Ultralytics YOLOv11, BoT-SORT / ByteTrack (fallback)
Disease classifier	PyTorch, ResNet50 (production model)
Backend API	Python 3.10, FastAPI, Uvicorn
Database	SQLite (`smartleaf.db`) via SQLAlchemy
Frontend	React 18 + TypeScript, Vite, Axios, Recharts

Table 3: Tech Stack

C.4 Repository layout (key paths only)

```
SmartLeafDetection/
|-- weights/                          # Trained model weights (see section 4)
|-- smart_leaf_detection/             # Reusable pipeline library
|                                       (device, classifier, config)
|-- training/                         # Training + evaluation pipelines
|   |-- leaf_detection/               # YOLO leaf detector training,
|   |                                   fix_resume_checkpoint.py
|   `-- disease_classification/       # ResNet50 / EfficientNet /
|                                       MobileNet + benchmark
|-- webapp/
|   |-- backend/                      # FastAPI app
|   |   |-- app/
|   |   |   |-- main.py               # API entry (app.main:app)
|   |   |   |-- worker.py             # The pipeline:
|   |   |   |                           detection -> track ->
|   |   |   |                           classify -> aggregate
|   |   |   |-- models.py             # DB tables
|   |   |   |-- database.py           # DB engine +
|   |   |   |                           ensure_columns() migration
|   |   |   |-- routes/               # auth_routes.py,
|   |   |   |                           flight_routes.py
|   |   |   `-- trackers/             # botsort_gmc.yaml,
|   |   |                               bytetrack.yaml
|   |   |-- compare_trackers.py       # Read-only A/B tracker
|   |   |                               comparison tool
|   |   `-- smartleaf.db              # SQLite database
|   `-- frontend/                     # React + Vite app
|-- scripts/                          # setup
|-- tests/                            # pytest test suite
`-- README.md                         # Full project history
```

C.5 Models and weights (`weights/`)

File	Role	Status
`leaf_best.pt`	YOLOv11 leaf detector (localization + tracking)	Production
`leaf_classifier.pt`	ResNet50 disease classifier ($\sim$0.82 validation Macro-F1)	Production
`best.pt`	Legacy YOLO disease detector	Fallback only
`benchmark/*.pt`	EfficientNetV2-S / MobileNetV3 / ResNet50 benchmark checkpoints	Experimentation
`leaf_baseline.pt`, `leaf_improved.pt`	Earlier leaf-detector iterations	Archive

Table 4: Model and weights

To swap the production classifier, either replace `weights/leaf_classifier.pt` or point the `CLASSIFIER_WEIGHTS` environment variable at another checkpoint (see Section C.8).

C.6 First-time setup

Prerequisites: Python 3.10, Node.js 18+, Git.

```
# From the project root

# macOS / Linux:
bash scripts/setup.sh

# Windows (PowerShell):
.\scripts\setup.ps1
```

This creates the virtual environment and installs the Python dependencies from `requirements.txt` and the frontend dependencies using `npm install`.

C.7 Running the system (two terminals)

Terminal 1 – backend The backend must be run from `webapp/backend`:

```
cd webapp/backend
uvicorn app.main:app --port 8000 --reload
```

Terminal 2 – frontend The frontend must be run from `webapp/frontend`:

```
cd webapp/frontend
npm run dev
```

Vite serves the frontend on `http://localhost:3000`.

Then open `http://localhost:3000` in a browser. The backend serves on `http://localhost:8000`; both must run at the same time.

Quick health checks With the backend running, use the following endpoints:

- `GET http://localhost:8000/api/health`

  Expected response:

  ```
  {"status": "ok"}
  ```

- `GET http://localhost:8000/api/model-info`

  This endpoint shows the active backend and classifier architecture. Verify that the classifier architecture is reported as `resnet50`.

C.8 Configuration – environment variables

Set the environment variables on the same line that starts the backend. For example:

```
FRAME_STRIDE_SEC=0.3 LEAF_TRACKER=botsort \
uvicorn app.main:app --port 8000
```

C.8 Configuration – environment variables

Variable	Default	Purpose
LEAF_TRACKER	botsort	Tracker backend: botsort (BoT-SORT with motion compensation) or bytetrack as fallback.
FRAME_STRIDE_SEC	0.7	Seconds between sampled frames. A lower value, such as 0.3, may improve tracking but requires more computation.
MIN_TRACK_LEN	1	A leaf must be detected in at least this number of frames before it becomes a final result.
LEAF_CONF	0.3	Confidence threshold used by the leaf detector.
DISEASE_BACKEND	auto	Classification backend: classifier (ResNet50), YOLO legacy model, or automatic selection.
CLASSIFIER_WEIGHTS	(auto)	Optional path override for the disease-classifier checkpoint.
MAX_LEAF_FRAMES	30	Maximum number of selected frames analyzed per video.
MIN_LEAVES	1	A frame must contain at least this number of leaf detections to be retained.
OUTPUT_DIR	processing Output	Directory in which extracted frames, leaf crops, and annotated images are written.

Table 5: Configuration – Environment Variables

Database credentials, the secret key, and storage paths are configured in webapp/.env.

C.9 Database

- **Type:** SQLite, using the file webapp/backend/smartleaf.db. For production-scale deployment, change DATABASE_URL to use MySQL.
- **Tables:** users, flights, plant_results (one row represents one tracked leaf), leaf_results (one row represents one observation or frame), and frame_records.
- **Migration:** The system does not use Alembic. database.ensure_columns() runs during startup and idempotently adds new columns to an existing database without data loss. When adding a column to models.py, the same column must also be added to ensure_columns().
- **Backup:** Back up the database by copying smartleaf.db while the server is stopped. Raw input videos stored in uploads/ should never be overwritten.

C.10 Common maintenance tasks

Switch the tracker: Set LEAF_TRACKER=bytetrack or LEAF_TRACKER=botsort. Tracker configuration files are located in:

```
webapp/backend/app/trackers/*.yaml
```

Relevant configuration parameters include `track_buffer` and `gmc_method`.

Compare trackers for evaluation or thesis evidence: Run both tracker configurations on the same video, and then execute:

```
cd webapp/backend
python compare_trackers.py
```

The script is read-only. It prints a comparison table and writes a CSV file under `reports/`.

Retrain the disease classifier:

```
python training/disease_classification/train_classifier.py \
  --arch resnet50
```

After training, promote the new checkpoint:

```
cp weights/benchmark/resnet50.pt weights/leaf_classifier.pt
```

Retrain or resume the leaf detector: Training scripts are located in:

```
training/leaf_detection/
```

If resuming an old checkpoint fails with the message `'cls_pw' is not a valid YOLO argument`, run:

```
python training/leaf_detection/fix_resume_checkpoint.py
```

This script removes stale configuration keys and creates backup files with the `.bak` extension.

Run the tests:

```
python -m pytest tests/ -v
```

C.11 Known limitations

- The `target_spot` class has low recall (~ 0.27). The model commonly confuses it with `septoria` and `powdery_mildew`. This is primarily a data limitation and was observed across all benchmarked architectures.
- Tracking performance depends on frame density. During fast UAV movement, an excessively large frame stride may fragment tracks even when BoT-SORT is used.
- Older flights created before the 2026-06-26 database migration do not contain the newer evidence fields and therefore display limited information. Re-running the analysis populates the missing fields.
- SQLite is suitable for a single-user environment or demonstration. MySQL is recommended for concurrent production use.